Lesson 12: Hearing It Happen

Unity

VIDEO GAME CREATION

This week: your game finally sounds like a game. Four moments - firing, hitting, clearing a wave, and losing - each get their own distinct cue.

Open your own [YourName]Game project from Lesson 11 - don't start a new one.

If you already built an optional sound idea in an earlier lesson (Lessons 2, 3, 4, or 6), keep it - today reuses it directly instead of replacing it. If you didn't, today's steps are written as if starting fresh.

Setup

1. Open your [YourName]Game project from Lesson 11.
2. Add an AudioSource component to the GameManager GameObject itself (Play On Awake unchecked) - one shared player for win/wave/lose sounds.
3. Import sfx_wave.wav and sfx_lose.wav alongside your existing clips.

Level 1 - Minimal (MVP)

Four sounds fire at the correct moment, at least once each: a shot, a hit/explosion, a wave-clear (on any non-final wave), and a game over. No silence, no obviously wrong triggers.

4. In GameManager.cs, add public AudioSource roundAudioSrc and public AudioClip winClip, waveClip, loseClip fields, then drag the AudioSource component and the three clips onto them in the Inspector.
5. If GameManager doesn't already expose itself via a public static GameManager instance (assigned in Awake()), add that now - the same singleton pattern Lesson 10 introduced for EnemySpawner, needed here so the static RegisterKill()/RegisterEscape() methods can reach the Inspector-assigned AudioSource.
6. Confirm the Player's AudioSource already plays sfx_shoot.wav on fire (from Lesson 3) - add it now if it's missing.
7. Confirm the Enemy prefab's AudioSource already plays sfx_hit.wav on destroy (from Lesson 4) - add it now if it's missing.
8. In RegisterKill()'s nested if (from Lesson 11), call instance.roundAudioSrc.PlayOneShot() with winClip in the true-win branch and waveClip in the wave-advance branch.
9. In RegisterEscape()'s lose branch, call instance.roundAudioSrc.PlayOneShot(instance.loseClip).

10. Press Play and play through a full round - shoot, hit, an early wave clear, and a loss should each sound distinct.

```
// GameManager.cs:
public static GameManager instance;
public AudioSource roundAudioSrc;
public AudioClip winClip, waveClip, loseClip;

void Awake() { instance = this; }

public static void RegisterKill() {
    enemiesRemaining--;
    score += 10;
    if (enemiesRemaining <= 0) {
        if (wave >= totalWaves) {
            hasWon = true;
            instance.roundAudioSrc.PlayOneShot(instance.winClip);
        } else {
            wave++;
            enemiesRemaining = 5;
            instance.roundAudioSrc.PlayOneShot(instance.waveClip);
            EnemySpawner.instance.SpawnEnemies();
        }
    }
}

public static void RegisterEscape() {
    lives--;
    if (lives <= 0) {
        hasLost = true;
        instance.roundAudioSrc.PlayOneShot(instance.loseClip);
    }
}
```

Alternate Path: Visual Scripting

The same three-clip pattern works in a Script Graph instead of C#.

11. On the win/wave Branch node from Lesson 11, add an Audio Play Clip One Shot node to each outcome, referencing a winClip/waveClip variable.
12. On the lose graph, add a matching Audio Play Clip One Shot node referencing a loseClip variable.

Level 2 - Complete (Alpha/Beta)

All four sounds fire reliably every time, with no missing or duplicate playback, and each is clearly distinguishable from the others by ear.

- Play a full round: shoot, hit, an early wave clear, and a loss each produce a distinct sound.
- If you built the optional win sound, confirm it still plays on the true win, and waveClip doesn't also fire on that same frame.
- PlayOneShot() lets overlapping sounds layer instead of cutting each other off - confirm rapid shooting and a hit landing together both play correctly.

Level 3 - Refined (Ready to Publish) - pick any that interest you

- Audio: if you haven't already, add the optional win sound now - it slots into the exact same PlayOneShot() pattern as today's wave/lose sounds, just a third clip.
- Audio: add slight pitch variation to hits by setting the Enemy's AudioSource.pitch to a random value (e.g. Random.Range(0.9f, 1.1f)) right before Play().
- Structure: notice roundAudioSrc is shared by three different outcomes (win, wave-clear, lose) via PlayOneShot() - one AudioSource, three interchangeable clips.

If Something's Wrong

- NullReferenceException on instance.roundAudioSrc: confirm GameManager's Awake() actually runs before any RegisterKill()/RegisterEscape() call, and that instance is assigned there.
- No sound on wave-clear: confirm the waveClip PlayOneShot() call is actually inside the else branch, not after the if/else.
- Lose sound plays repeatedly: confirm the lose branch is still guarded by "if (lives <= 0)" as a one-time check (from Lesson 8), not re-triggering every frame.

Want More?

Student Resource List > Unity section, or learn.unity.com